



操作系统

习题课

刘文彬

一. 作业

二. 模拟题

共5次：

一、并发与处理机调度

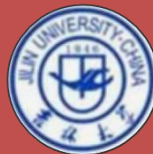
二、同步互斥

三、死锁1

四、死锁2，虚拟存储

五、文件与设备

一、并发与处理机调度

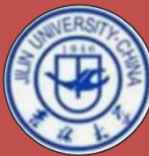


1. 进程P对磁盘数据进行处理，其处理流程为：① **读入**一块磁盘数据；② **处理**该块数据；③ 把处理后的该块数据**写到**磁带上。

假设：进程P要处理3个磁盘块的数据，读一个磁盘块数据需要40ms，处理一块数据需要20ms，写一块磁带数据需要80ms，并假设系统只有进程P运行。在不考虑系统开销的情况下，

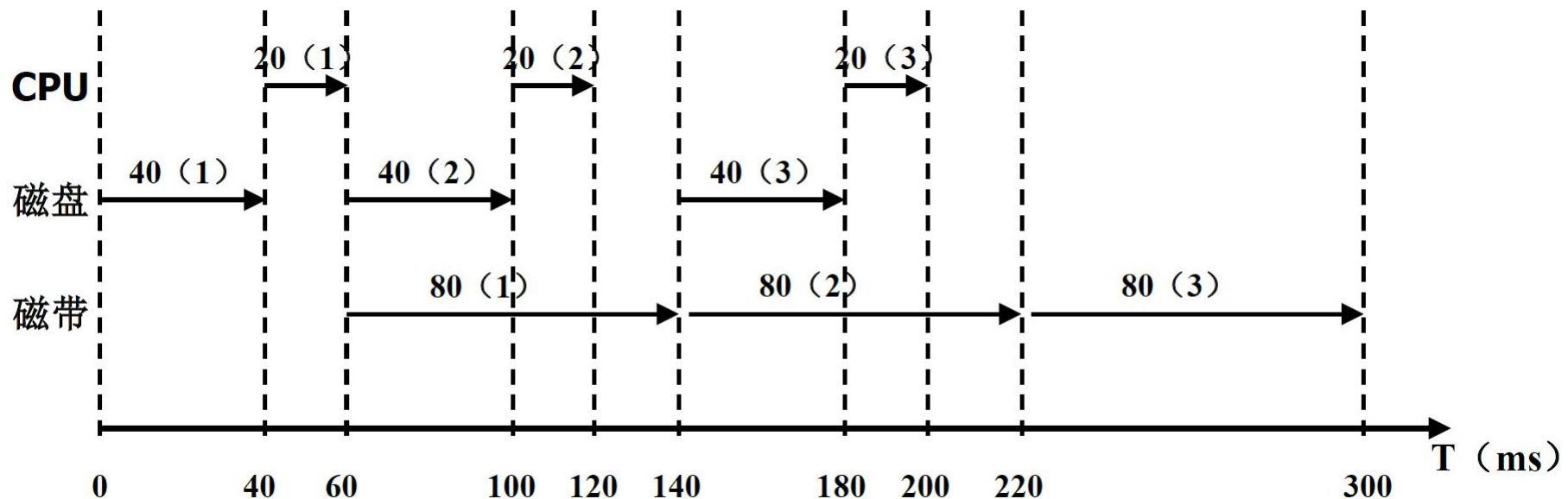
问题：

- (1) 进程P把3块数据处理流程都完成，画出**CPU**运行、**磁盘**输入、**磁带**输出的时序图；
- (2) 依据(1)画出的时序图，计算进程P等待设备I/O的时间。



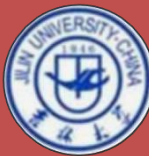
一、并发与处理机调度

答：(1) 进程P把3块数据处理流程都完成，画出**CPU**运行、**磁盘**输入、**磁带**输出的时序图；



(2) 依据(1)画出的时序图，计算进程P等待设备I/O的时间。

进程P等待设备I/O的时间 = $40 + 40 + 20 + 40 + 20 = 160$ (ms)。



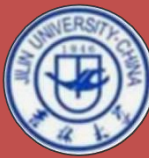
一、并发与处理机调度

2. 某系统按最短剩余时间（Shortest Remaining Time First）优先算法调度CPU。已知四个进程P1、P2、P3、P4的到达时间和CPU阵发时间如下（时间单位：毫秒）：

Process	Arrival time	Burst time
P1	0	10
P2	3	6
P3	4	3
P4	7	4

问题：

- (1) 画出按最短剩余时间优先的CPU调度算法得到的进程调度结果的Gantt图；
- (2) 计算四个进程的平均等待时间、平均周转时间、平均带权周转时间。

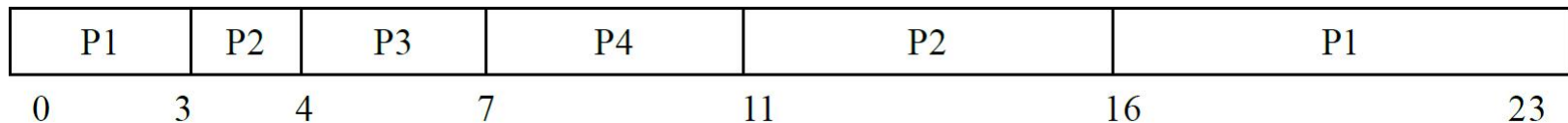


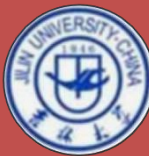
一、并发与处理机调度

Process	Arrival time	Burst time
P1	0	10
P2	3	6
P3	4	3
P4	7	4

答：(1) 画出按最短剩余时间优先的CPU调度算法得到的进程调度结果的Gantt图；

- ① P1:0-3, P1' =7; P2' =6;
- ② P2:3-4, P1' =7; P2' =5; P3' =3
- ③ P3:4-7, P1' =7; P2' =5; P3' =0; P4' =4
- ④ P4:7-11;



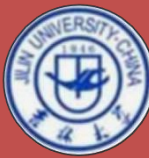


一、并发与处理机调度

P1	P2	P3	P4	P2	P1	
0	3	4	7	11	16	23

(2) 平均等待时间、平均周转时间、平均带权周转时间；

进程	到达时间	运行时间	开始时间	完成时间	等待时间	周转时间	带权 周转时间
P1	0	10	0	23	13	23	23/10
P2	3	6	3	16	7	13	13/6
P3	4	3	4	7	0	3	1
P4	7	4	7	11	0	4	1
平均等待时间 = $(13+7+0+0) / 4 = 20/4 = 5 \text{ ms}$							
平均周转时间 = $(23+13+3+4) / 4 = 43/4 = 10.75 \text{ ms}$							
平均带权周转时间 = $(23/10+13/6+1+1) / 4 = 194/(30*4) \approx 1.62 \text{ ms}$							



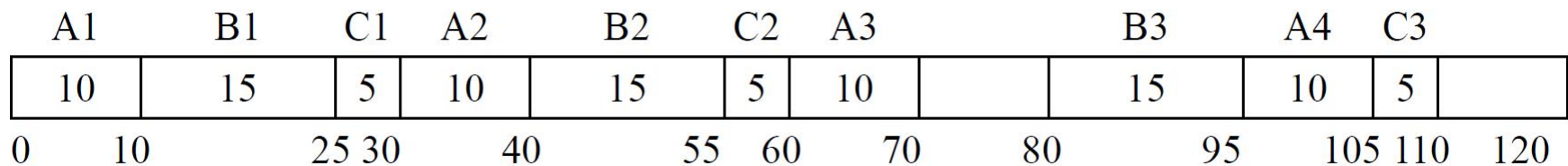
一、并发与处理机调度

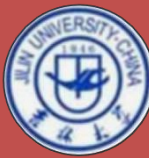
3. 设有周期性实时任务集合如下表所示，用EDF算法是否可以调度？画出前100个时间单位的Gantt图。

任务	发生周期 T_i	处理时间 C_i
A	30	10
B	40	15
C	50	5

最早截止期优先 (earliest deadline first, EDF)

$$\sum \frac{C_i}{T_i} = \frac{10}{30} + \frac{15}{40} + \frac{5}{50} = \frac{97}{120} = 0.808 < 1, \text{因而EDF算法可以调度}$$





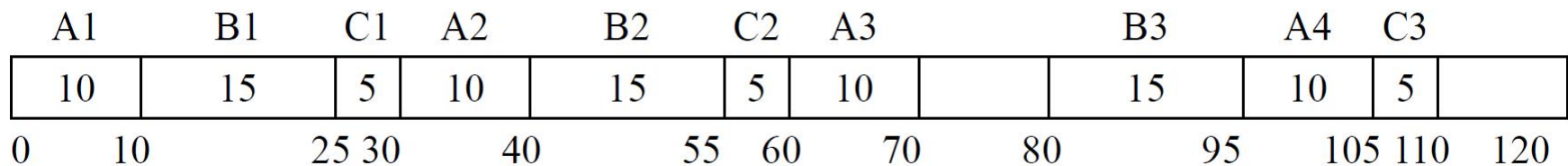
一、并发与处理机调度

3. 设有周期性实时任务集合如下表所示，用EDF算法是否可以调度？画出前100个时间单位的Gantt图。

任务	发生周期 T_i	处理时间 C_i
A	30	10
B	40	15
C	50	5

答：最早截止期优先 (earliest deadline first, EDF)

$$\sum \frac{C_i}{T_i} = \frac{10}{30} + \frac{15}{40} + \frac{5}{50} = \frac{97}{120} = 0.808 < 1, \text{因而EDF算法可以调度}$$



二、同步互斥



```
1.  var blocked: array[0..1] of boolean;  
    turn: 0..1;  
    procedure P(id: integer);  
    begin  
        repeat  
            blocked[id] := true;  
            while turn <> id do  
                begin  
                    while blocked[1-id] do  
                        {nothing}  
                *   turn := id  
                end;  
            <Critical section>  
            blocked[id] := false;  
            <Remainder>  
        until false  
    end;
```

```
begin  
    blocked[0] := false;  
    blocked[1] := false;  
    turn := 0;  
    parbegin  
        P(0); P(1)  
    parend;  
end.
```

Find a counter example to demonstrate that this solution is incorrect.

二、同步互斥



回顾：实现进程互斥（临界区的管理），满足3个正确性原则

1. mutual exclusion(互斥性)：一次只允许一个进程进入关于同一组公共变量的临界区；
2. Progress(进展性)：临界区空闲时，放行一个进入者；
3. bounded waiting(有限等待性)：一个想要进入临界区的进程在等待有限个进程进入并离开临界区后获得进入临界区的机会。

二、同步互斥



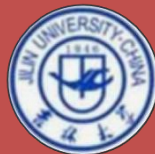
```
1.  var blocked: array[0..1] of boolean;
      turn: 0..1;
      procedure P(id: integer);
      begin
        repeat
          blocked[id] := true;
          while turn <> id do
            begin
              while blocked[1-id] do
                {nothing}
              *   turn := id;
            end;
          <Critical section>
          blocked[id] := false;
          <Remainder>
        until false
      end;
```

```
begin
  blocked[0] := false; blocked[1] := false;
  turn := 0;
  parbegin
    P(0); P(1)
  parend;
end.
```

(1) 不满足互斥性：若 $turn=1$, $P(0)$ 先推进至*位置时发生进程切换, $P(1)$ 占据处理机, 就会出现 $P(0)$ 和 $P(1)$ 进程均进入临界区;

(2) 不满足有限等待性, 若 $P(0)$ 进程先推进, 且进入临界区, $P(1)$ 进程会在第二个 `while` 循环处忙式等待, 若推进快, 就会造成 $P(1)$ 长时间不能进入临界区。

二、同步互斥

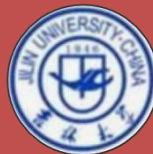


2. 请分析管程实现读者写者问题时startread () 中signal(rq)的作用?

① wait(c): 如果紧急等待队列非空, 则唤醒第一个等待者, 否则释放管程的互斥权。执行此操作进程的进程控制块进入c链的尾部。

② signal(c): 如果c链为空, 则相当于空操作, 执行此操作的进程继续; 否则唤醒第一个等待者, 执行此操作进程的进程控制块进入紧急等待队列的尾部。

二、同步互斥

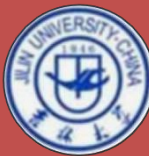


```
procedure start_read;
begin
    if write_count>0 then
        wait(rq);
    reading_count:=reading_count+1;
    signal(rq);
end;

procedure finish_read;
begin
    reading_count:=reading_count-1;
    if reading_count=0 then
        signal(wq);
    end;
```

```
procedure start_write;
begin
    write_count:=write_count+1;
    if(write_count>1)or(reading_count>0) then
        wait(wq);
    end;
procedure finish_write;
begin
    write_count:=write_count-1;
    if write_count>0 then
        signal(wq)
    else
        signal(rq);
    end;
```

答：如果没有startread（）中signal(rq)，就会出现等待写者的第一个读者被最后一个写者唤醒，但是rq中的其它读者得等下一批写者中的最后一个写者离开时才能被唤醒，而且也只能唤醒一个。



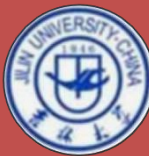
三、死锁1

1. 在银行家算法中，出现如下资源分配情况：

	<u>Allocation</u>				<u>Need</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P0:	0	0	3	2	0	0	1	2	1	6	2	3
p1:	1	0	0	0	1	7	5	0				
p2:	1	3	5	4	2	3	5	6				
p3:	0	3	3	2	0	6	5	2				
p4:	0	0	1	4	0	6	5	6				

(1) 当前状态是否安全？

(2) 如果进程P2提出Request[2]=(1, 2, 2, 2)，系统能否将资源分配给它？试说明原因。



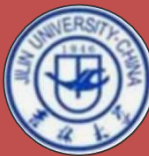
三、死锁1

	<u>Allocation</u>				<u>Need</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P0:	0	0	3	2	0	0	1	2	1	6	2	3
p1:	1	0	0	0	1	7	5	0				
p2:	1	3	5	4	2	3	5	6				
p3:	0	3	3	2	0	6	5	2				
p4:	0	0	1	4	0	6	5	6				

答：（1）当前状态是安全的。运行安全检测算法如下：

- ① $Work = (1, 6, 2, 3)$, $Need[0] < work$, $work = work + allocation[0] = (1, 6, 5, 5)$, $Finish[0] = true$;
- ② $Need[3] < work$, $work = (1, 9, 8, 7)$, $Finish[3] = true$;
- ③ $Need[4] < work$, $work = (1, 9, 9, 11)$, $Finish[4] = true$;
- ④ $Need[1] < work$, $work = (2, 9, 9, 11)$, $Finish[1] = true$;
- ⑤ $Need[2] < work$, $work = (3, 12, 14, 15)$, $Finish[2] = true$;

安全进程序列 $\langle P0, 3, 4, 1, 2 \rangle$, 对于所有 i , $Finish[i] = true$, 系统安全。



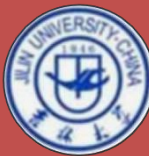
三、死锁1

	<u>Allocation</u>				<u>Need</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P0:	0	0	3	2	0	0	1	2	1	6	2	3
p1:	1	0	0	0	1	7	5	0				
p2:	1	3	5	4	2	3	5	6				
p3:	0	3	3	2	0	6	5	2				
p4:	0	0	1	4	0	6	5	6				

答： (2) Request[2]=(1, 2, 2, 2) < Need[2]=(2, 3, 5, 6), 请求合法;

Request[2]=(1, 2, 2, 2) < Available=(1, 6, 2, 3), 请求可以满足。假设分配资源, 则系统状态变为

		<u>Allocation</u>				<u>Need</u>				<u>Available</u>			
		A	B	C	D	A	B	C	D	A	B	C	D
运行安全性检测，	P0:	0	0	3	2	0	0	1	2	0	4	0	1
不存在安全进程	p1:	1	0	0	0	1	7	5	0				
	p2:	2	5	7	6	1	1	3	4				
序列，取消分配。	p3:	0	3	3	2	0	6	5	2				
	p4:	0	0	1	4	0	6	5	6				



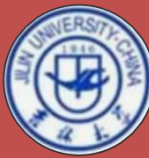
三、死锁1

2. 某系统采用死锁检测手段以发现死锁，设系统中的资源类集合为 {A, B, C}，资源A中共有8个实例，B中有6个，C中有5个。又设系统中的进程集合为 {P1, 2, 3, 4, 5, 6}，某时刻系统状态如下：

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
p1:	1	0	0	0	0	0	2	2	1
p2:	3	2	1	0	0	0			
p3:	0	1	2	2	0	2			
p4:	0	0	0	0	0	0			
p5:	2	1	0	0	3	1			
p6:	0	0	1	0	0	0			

(1) 上述状态下，系统依次接收如下请求：Request[1]=(1, 0, 0)，Request[2]=(2, 1, 0)，Request[4]=(0, 0, 2)。给出系统状态变化的情况，并说明没有死锁。

(2) 在(1)确定的状态下，系统接收Request[1]=(0, 3, 1)，说明此时易发生死锁，并找出参与死锁的进程。



三、死锁1

2.

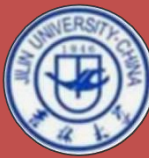
	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
p1:	1	0	0	0	0	0	2	2	1
p2:	3	2	1	0	0	0			
p3:	0	1	2	2	0	2			
p4:	0	0	0	0	0	0			
p5:	2	1	0	0	3	1			
p6:	0	0	1	0	0	0			

答：（1）如果仅接收请求，但未分配资源，则状态变为

①

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
p1:	1	0	0	1	0	0	2	2	1
p2:	3	2	1	2	1	0			
p3:	0	1	2	2	0	2			
p4:	<u>0</u>	<u>0</u>	<u>0</u>	0	0	2			
p5:	2	1	0	0	3	1			
p6:	0	0	1	0	0	0			

运行死锁检测算法，可找到安全进程序列<P4, 1, 2, 3, 5, 6>，系统没有进入死锁状态。



三、死锁1

(1)

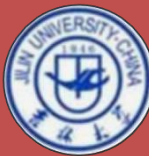
	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
p1:	1 0 0	1 0 0	2 2 1
p2:	3 2 1	2 1 0	
p3:	0 1 2	2 0 2	
p4:	0 0 0	0 0 2	
p5:	2 1 0	0 3 1	
p6:	0 0 1	0 0 0	

如果接收请求后，将1个A分配给P1，则状态变为

①

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
p1:	2 0 0	0 0 0	1 2 1
p2:	3 2 1	2 1 0	
p3:	0 1 2	2 0 2	
p4:	0 0 0	0 0 2	
p5:	2 1 0	0 3 1	
p6:	0 0 1	0 0 0	

运行死锁检测算法，可找到安全进程序列<P4, 1, 2, 3, 5, 6>，系统没有进入死锁状态。



三、死锁1

(2) 在①状态下, 接收Request[1]=(0, 3, 1), 则系统状态变为

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
p1:	1	0	0	1	3	1	2	2	1
p2:	3	2	1	2	1	0			
p3:	0	1	2	2	0	2			
p4:	0	0	0	0	0	2			
p5:	2	1	0	0	3	1			
p6:	0	0	1	0	0	0			

运行死锁检测算法, 可找到安全进程序列<P4, 2, 3, 5, 6, 1>, 系统没有进入死锁状态。

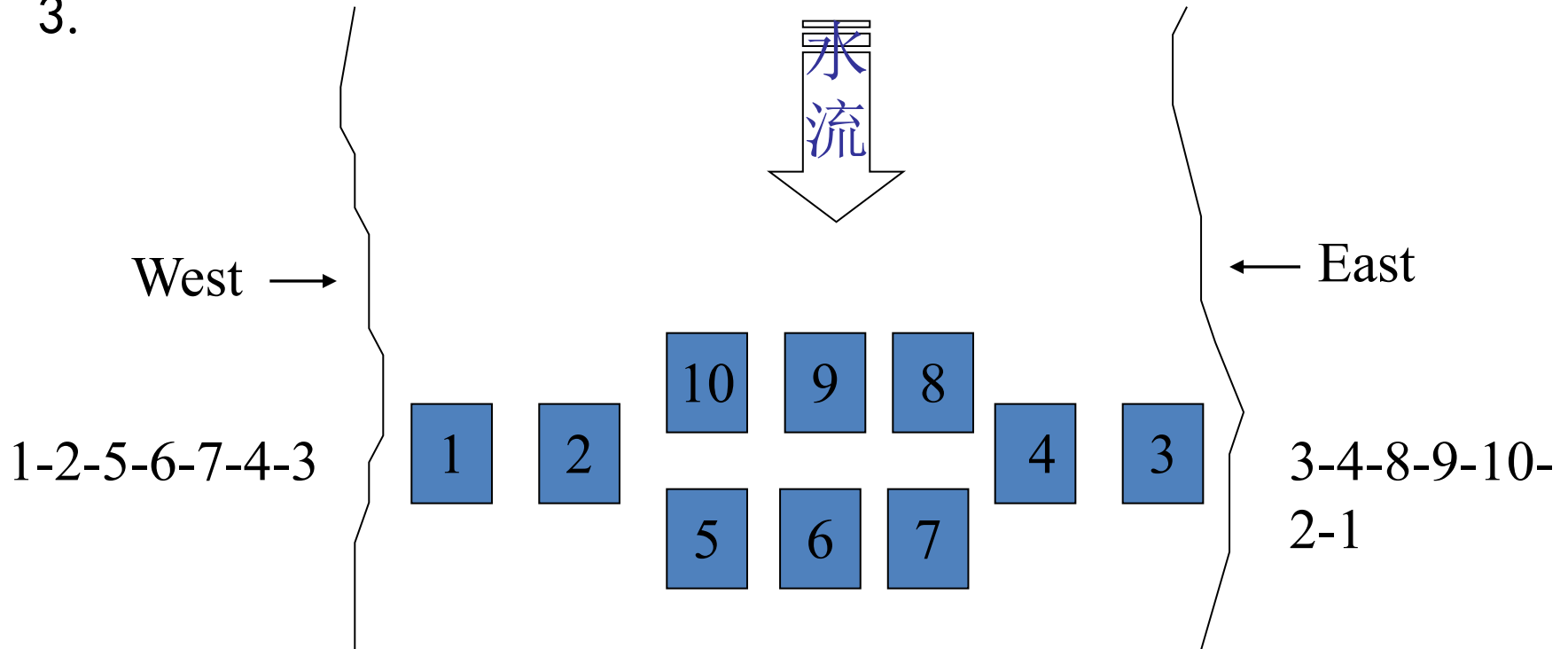
在②状态下, 接收Request[1]=(0, 3, 1), 则系统状态变为

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
p1:	2	0	0	0	3	1	1	2	1
p2:	3	2	1	2	1	0			
p3:	0	1	2	2	0	2			
p4:	0	0	0	0	0	2			
p5:	2	1	0	0	3	1			
p6:	0	0	1	0	0	0			

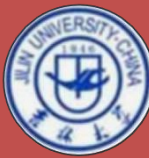
运行死锁检测算法, 找不到安全序列, 系统进入死锁状态。

三、死锁1

3.



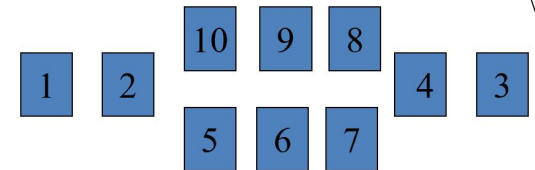
分析可能发生死锁的情况；设计无死锁、无饿死、并行度高的解法，用PV操作实现。



三、死锁1

3. 答：死锁情况：当一位西岸踏上1，一位东岸踏上2，形成死锁；3/4类似。当四位西岸过河者踏上2567，四位东岸48910，形成死锁。

对1, 2/3, 4可采用有序分配法防止死锁。限制同时过河人数不超过7人，可防止死锁。



Semaphore Smax; //初值=7

Semaphore s1, s2, s3, s4, s5, s6, s7, s8, s9, s10; //初值=1

West:

P(Smax); P(s1); do(s1); P(s2); do(s2); V(s1); P(s5); do(s5); V(s2); P(s6); do(s6); V(s5); P(s7); do(s7); V(s6);

P(s3); P(s4); (有序申请) do(s4); V(s7); do(s3); V(s4);
do(East); V(s3); V(Smax)



三、死锁1

4. 设有7个简单资源：A~G, 其申请命令为a~g, 释放命令为a'~g' ;系统中有P1, P2, P3, 三个进程, 其活动分别列举如下:

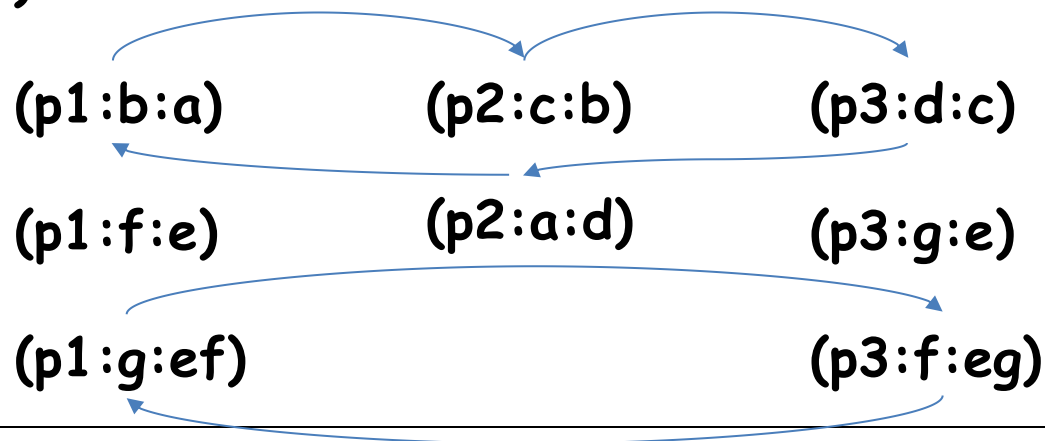
P1: a b a' b' e f g e' f' g'

P2: b c b' c' d a d' a'

P3: c d c' d' e g f e' f' g'

分析他们并发执行时, 是否有死锁可能性, 并说明原因。

答: 按照可重用组合资源死锁的静态分析算法画出进程占有、等待资源图。
(进程:请求:占有)

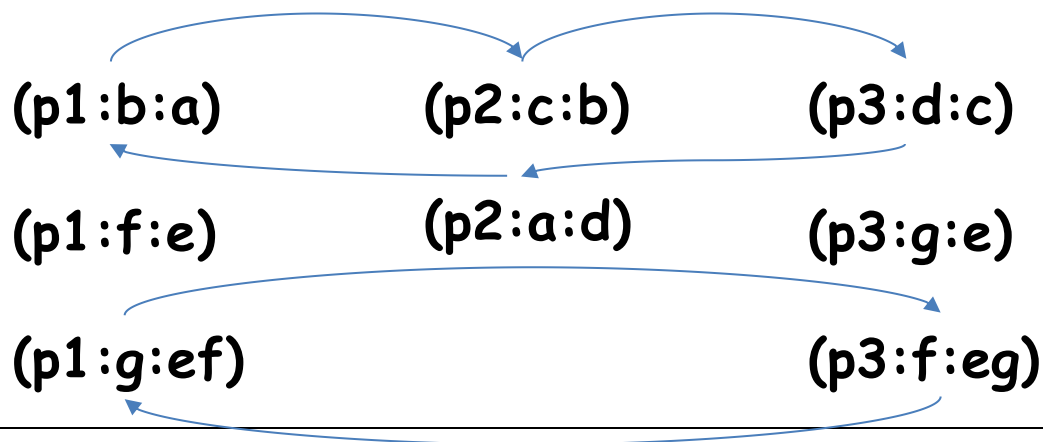


三、死锁1

包含2个环路：

- ① 上面这个环路，包含相同进程P2，由于P2不可能处于两种状态，因此环路不能形成；
- ② 下面这个环路，包含相同的被占资源e，由于同一资源不能同时分给两个进程，因此环路不能形成。

综上，3个进程并发执行时，没有死锁可能。





四、死锁2，虚拟存储

1. 制盒生产线上有一箱子，其中有 n 个位置 ($n \geq 2$)，每个位置可放一盒体或一盒盖，又设有三个工人P1, P2, P3其活动分别为：

P1:

P2:

P3:

加工一盒体；

加工一个盒盖；

箱中取一盒体；

盒体放入箱中；

盒盖放入箱中；

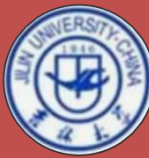
箱中取一盒盖；

组成一盒子；

(1) 信号量和P、V操作实现三个工人的合作，要求无死锁。

(2) 管程实现三个工人的合作

(3) 会实现三个工人的合作



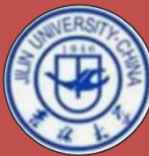
四、死锁2，虚拟存储

答：死锁：当所有箱子都放了盒体，P3无法组成盒子；当都放了盒盖，P3也无法组成盒子。

为了防止死锁，限制箱子中盒体不可超过 $n-1$ ，盒盖同理。

Semaphore $s1=n-1$; Semaphore $s2=n-1$;

工人1:	工人2:	工人3:
Do {	Do {	Do {
加工盒体;	加工盒盖;	P(body);取盒体; V(empty);
P(s1);	P(s2);	V(s1);
P(empty);	P(empty);	P(lid);取盒盖; V(empty);
入箱;	入箱;	V(s2);
V(body);	V(lid);	组装;
}	}	}



四、死锁2，虚拟存储

2. 某系统采用虚拟页式存储管理方式，页面大小为2KB，每个进程分配的页框数固定为4页。采用局部置换策略，置换算法采用改进的时钟算法，当有页面新装入内存时，页表的时钟指针指向新装入页面的下一个在内存的表项。设当前进程P的页表如下（“时钟”指针指向逻辑页面3的表项）：

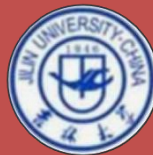
(1) 当进程P依次对逻辑地址执行下述操作：

① 引用 4C7H； ② 修改 19B4H； ③ 修改 0C9AH；

写出进程P的页表内容；

(2) 在(1)的基础上，当P对逻辑地址27A8H进行访问，该逻辑地址对应的物理地址是多少？

逻辑页号	页框号	访问位 r	修改位 m	内外标识
0	101H	0	0	1
1	—			0
2	110H	1	0	1
→ 3	138H	0	0	1
4	—			0
5	100H	1	1	1



四、死锁2，虚拟存储

(1) 当进程P依次对逻辑地址执行下述操作：

① 引用 4C7H； ② 修改 19B4H； ③ 修改 0C9AH；

写出进程P的页表内容；

页面大小为 2KB， $2KB=2 \times 2^{10}=2^{11}$ ，

即逻辑地址和物理地址的地址编码的低 11 位为页内偏移；

(1) ① 逻辑地址 4C7H=0100 1100 0111B，高于 11 位为 0，所以该地址访问逻辑页面 0；

引用 4C7H，页表表项 0：r=1；

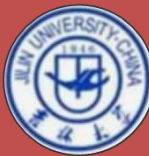
② 逻辑地址 19B4H=0001 1001 1011 0100B，高于 11 位为 3，所以该地址访问逻辑页面 3；修改 19B4H，页表表项 3：r=1, m=1；

③ 逻辑地址 0C9AH=0000 1100 1001 1010B，高于 11 位为 1，所以该地址访问逻辑页面 1；逻辑页 1 不在内存，发生缺页中断；

按改进的时钟算法，且时钟指针指向表项 3，应淘汰 0 页面，

即把 P 的逻辑页面 1 读到内存页框 101H，页表时钟指针指向表项 2。

并执行操作： 修改 0C9AH。



四、死锁2，虚拟存储

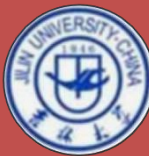
(1) 当进程P依次对逻辑地址执行下述操作：

① 引用 4C7H； ② 修改 19B4H； ③ 修改 0C9AH；

写出进程P的页表内容；

经上述 3 个操作后，P 的页表如下：

逻辑页号	页框号	访问位 r	修改位 m	内外标识
0	—	0	0	0
1	101H	1	1	1
→ 2	110H	0	0	1
3	138H	0	1	1
4	—			0
5	100H	0	1	1



四、死锁2，虚拟存储

(2) 在 (1) 的基础上，当P对逻辑地址27A8H进行访问，该逻辑地址对应的物理地址是多少？

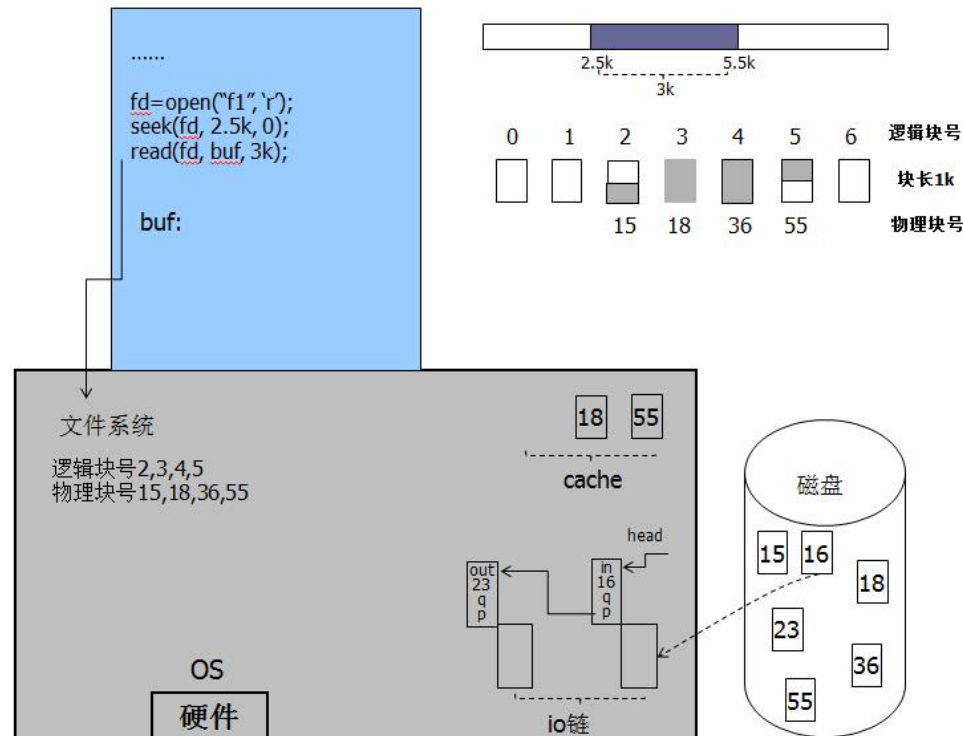
逻辑地址 27A8H=0010 0111 1010 1000B，高于 11 位为 4，所以该地址访问逻辑页面 4；页面 4 不在内存，发生缺页中断；按改进的时钟算法，淘汰页面 2，页面 4 读到 110H 页框，

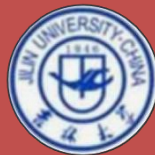
所以，逻辑地址 27A8H 对应的物理地址为：

0001 0001 0000 111 1010 1000B=887A8H。

五、文件与设备

1. 某进程依次执行open, seek, read三个系统调用读取文件f1中自2,5K起始的3K字节数据, 假定磁盘块大小为1K, 文件逻辑块号与物理块号的对应关系, 以及磁盘设备IO链、缓存链如下图所示, 试用文字描述read命令将所需信息读入进程空间的完整步骤, 并画出传输结束时的进程空间, 磁盘io链, 磁盘缓存链示意图。



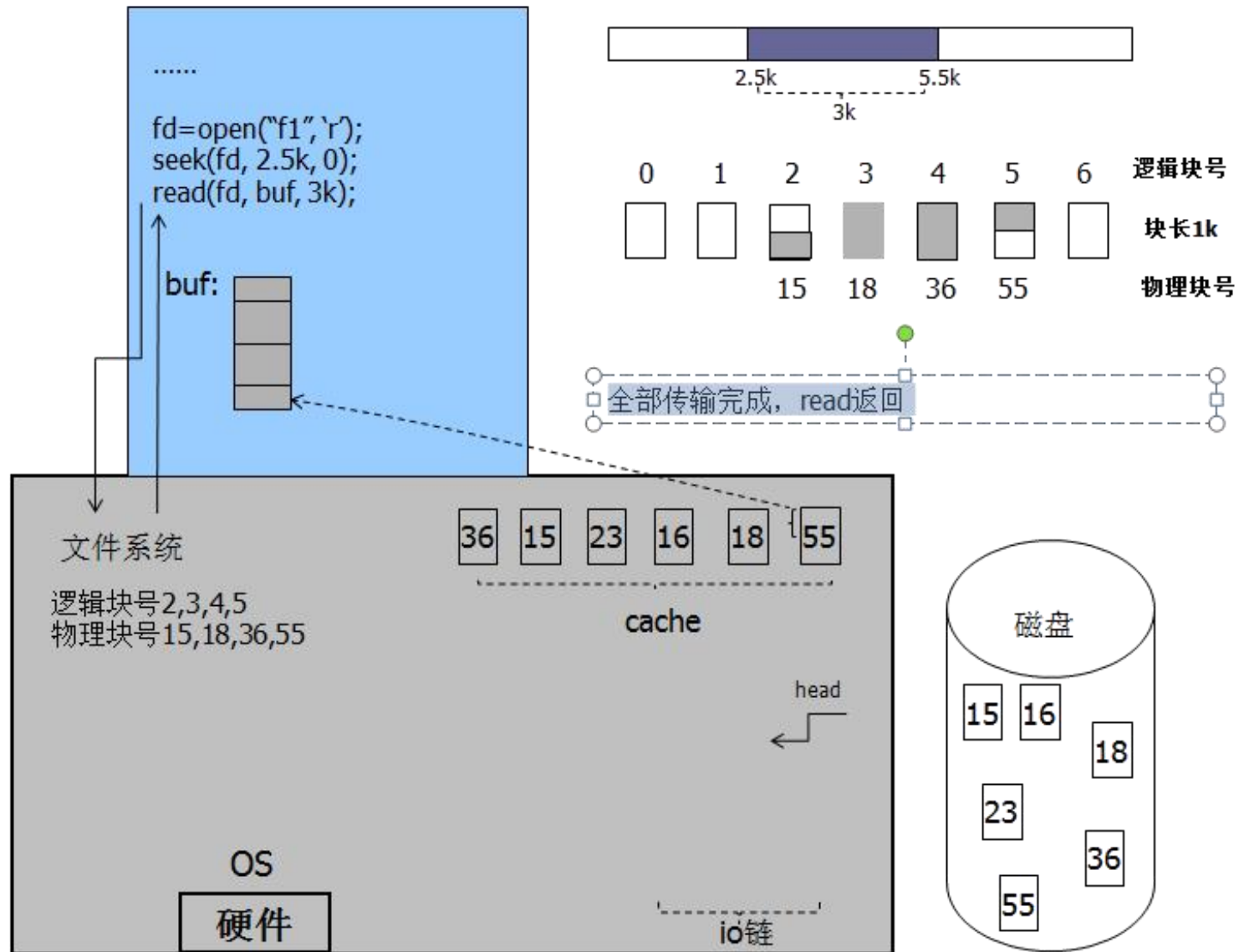


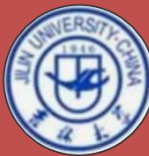
五、文件与设备

答：（15分）

1. 根据当前指针和访问量确定待访问逻辑块2, 3, 4, 5
2. 根据逻辑块和物理结构确定待访问物理块15, 18, 36, 55
3. 第15块不在cache, 分配缓冲, 填写头部, 入io链, 等待在q上
4. 第16块io完成, 发生中断, 唤醒相关进程, 信息复制到相关进程空间后, 缓冲入cache, 启动io链上下一块传输
5. io传输完, 发生中断, 缓冲入cache, 开始下块传输
6. io传输完, 发生中断, 唤醒本进程
7. 15块上需要信息复制到进程空间, 15送cache
8. 第18块在cache, 直接复制到进程空间
9. 第36块不在cache, 分配缓冲区, 填写头部, 入io链, 启动设备, 开始传输36块, 等待q上
10. 传输完成, 发生中断, 唤醒q上等待的本进程
11. 获得处理机后, 36块信息复制进程空间
12. 缓冲区送cache
13. 第55块在cache, 前半部分复制到进程空间
14. 全部传输完成, read返回

五、文件与设备



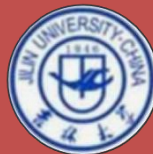


五、文件与设备

2. 设磁盘磁道由外向内编号为：0、1、2、……、199；磁头移动一个磁道所需时间为1ms；每个磁道有 32 个扇区；磁盘转速 $R=6000\text{r/min}$. 系统对磁盘设备的I/O请求采用冻结电梯算法 (Freezing Look, FLOOK) 调度算法，每个请求读/写磁道上的1个扇区。当前磁头向内移动，从20ms时刻开始处理当前服务队列，并且在120ms时刻处理完该队列的最后一个110磁道的I/O请求。

设有磁道的I/O请求序列如下表所示：

磁 道	60	110	40	102	115	110	70	30	60	40
到达时间 (ms)	23	40	65	85	105	125	145	165	185	190



五、文件与设备

问 题：(1) 写出表中I/O请求序列的调度序列，并计算磁头的移动量；

(2) 对表中给定I/O请求序列，计算：总寻道时间（忽略启动时间）、总旋转延迟时间、总传输时间和总访问处理时间。

磁 道	60	110	40	102	115	110	70	30	60	40
到达时间(ms)	23	40	65	85	105	125	145	165	185	190

(1)调度序列为：110→115→102→60→40→30→60→70→110

$$\begin{aligned}\text{磁头移动量} &= (115-110) + (115-102) + (102-60) + (60-40) + (40-30) + (60-30) + (70-60) + (110-70) \\ &= 5 + 13 + 42 + 20 + 10 + 10 + 30 + 10 + 40 = 180 \text{ (磁道)}\end{aligned}$$

(2)总寻道时间 $=1 \times 180 = 180 \text{ (ms)}$

$$\text{一次访盘的旋转延迟时间} = 1 / (2R) = 1 / (2 \times 6000 / \text{min}) = 1 / (2 \times (100 / \text{s})) = 0.005 \text{ (秒)} = 5 \text{ (ms)}$$

$$\text{请求序列共 10 次访盘，总旋转延迟时间} = 5 \times 10 = 50 \text{ (ms)}$$

$$\text{1 次访盘的传输时间} = 1 / (R \times 32) = 1 / ((6000 / \text{min}) \times 32) = 10 / 32 = 5 / 16 \text{ (ms)}$$

$$\text{10 次访盘总传输时间} = 5 / 16 \times 10 = 25 / 8 = 3.125 \text{ (ms)}$$

$$\text{总访盘处理时间} = 180 + 50 + 3.125 = 233.125 \text{ (ms)}$$



一. 作业

二. 模拟题

共五项：

一、单选：20小题，共40 分

二、实时调度： 15 分

三、死锁： 15 分

四、存储管理： 15 分

五、信号量和 PV 操作： 15 分

一、单选



1. 操作系统提供给编程人员的接口是 **C**。
A 库函数 B 高级语言 C 系统调用 D 子程序

2. 在中断发生后，进入中断处理程序属于 **C**。
A 用户程序 B 可能是应用程序，也可能是操作系统程序
C 操作系统程序 D 既不是应用程序，也不是操作系统程序

3. 进程从运行态到等待态可能是 **A**。
A 运行进程执行P 操作 B 进程调度程序的调度
C 运行进程的时间片用完 D 运行进程执行了V 操作

一、单选



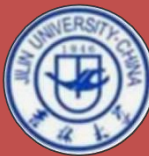
4. 下面哪些特征是并发程序执行的特点 **A** 。

- A 程序执行的间断性 B 相互通信的可能性
C 产生死锁的可能性 D 资源分配的动态性

5. 现有 3 个同时到达的作业J1、J2、J3，它们的执行时间分别是T1、T2 和T3，且 $T1 < T2 < T3$ ，若系统按单道方式运行且采用作业优先调度算法，则平均周转时间是 **B** 。

- A $T1+T2+T3$ B $(3T1+2T2+T3)/3$
C $(T1+T2+T3)/3$ D $(T1+2T2+3T3)/3$

一、单选



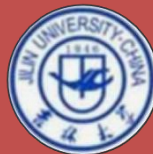
6. 在多进程的系统中，为了保证公共变量的完整性，各进程应互斥地进入临界区。所谓临界区是指 **D**。

- A 一个缓冲区
- B 一段数据区
- C 同步机制
- D 一段程序

7. 要实现两个进程互斥，设一个互斥信号量mutex，当mutex 为0 时，表示 **B**。

- A 没有进程进入临界区
- B 有一个进程进入临界区
- C 有一个进程进入临界区，另外一个进程在等候
- D 两个进程都进入临界区

一、单选



8. 即考虑作业等待时间，又考虑作业执行时间的调度算法是 **A**。

A 最高响应比优先调度算法

B 短作业优先调度算法

C 优先级调度算法

D 先来先服务调度算法

9. 死锁和安全状态的关系是 **D**。

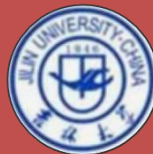
A 死锁状态有可能是安全状态 B 安全状态有可能成为死锁状态

C 不安全状态就是死锁状态 D 死锁状态一定是不安全状态

10. 设有 n 个进程共用一个相同的程序段，若每次最多允许 m 个进程 ($n \geq m$) 同时进入临界区，则信号量的初值为 **B**。

A n B m C $m-n$ D $-m$

一、单选



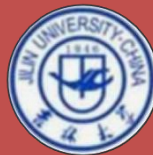
11. 一个进程被唤醒意味着 **A** 。

- A 该进程可以重新占有CPU
- B 优先级变为更大
- C PCB 移到就绪队列之首
- D 进程变为运行态

12. 下列关于进程和线程的叙述中，正确的是 **A** 。

- A 不管系统是否支持线程，进程都是资源分配的基本单位
- B 线程是资源分配的基本单位，进程是调度的基本单位
- C 系统级线程和用户级线程的切换都需要内核的支持
- D 同一进程中的各个线程拥有各自不同的地址空间

一、单选



13. 不会产生内部碎片的存储管理是 **B**。

A 页式存储管理

B 段式存储管理

C 固定分区式存储管理

D 段页式存储管理

14. 下面关于虚拟存储器的论述中，正确的是 **A**。

A 在段页式系统中以段为单位管理用户的逻辑地址空间，以页为单位管理内存的物理地址空间，有了虚拟存储器才允许用户使用比内存更大的地址空间

B 为了提高请求分页系统中内存的利用率，允许用户使用不同大小的页面

C 为了能让更多的作业同时运行，通常只装入10%-30%的作业即启动运行

D 最佳置换算法是实现虚拟存储器的常用算法

一、单选



15. 在可变分区分配管理中，某一作业完成后，系统收回其内存空间，并与相邻区合并，为此修改空闲区域表，造成空闲分区数减1的情况是 **D**。

- A 无上邻空闲分区，也无下邻空闲分区
- B 有上邻空闲分区，但无下邻空闲分区
- C 无上邻空闲分区，但有下邻空闲分区
- D 有上邻空闲分区，也有下邻空闲分区

一、单选



16. 操作系统采用分页存储管理方式，要求 **A**。

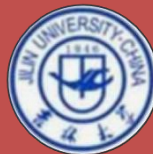
A 每个进程拥有一张页表，且进程的页表驻留在内存中

B 每个进程拥有一张页表，但只要执行进程的页表驻留在内存中

C 所有进程共享一张页表，以节约有限的内存空间，但页表必须驻留在内存中

D 所有进程共享一张页表，只有页表中当前使用的页面必须驻留在内存中

一、单选



17. 采用分段存储管理系统中，若段地址用24 位表示，其中8 位表示段号，则允许每段的最大长度为 **B**。

A $2^{24}B$ B $2^{16}B$ C 2^8B D $2^{32}B$

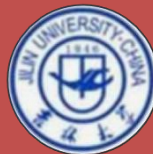
18. 在段页式分配中，CPU 每次从内存中取一次数据需要次访问内存。

A 1 B 2 C 3 D 4

19. 在Hoare 管程中，假设进程P 正在使用管程，当P 执行唤醒Signal 操作时，其含义是 **C**。

A Signal and continue B Signal and leave
C Signal and urgent wait D 以上都不对

一、单选



17. 采用分段存储管理系统中，若段地址用24 位表示，其中8 位表示段号，则允许每段的最大长度为 **B** 。

A $2^{24}B$ B $2^{16}B$ C 2^8B D $2^{32}B$

18. 在段页式分配中，CPU 每次从内存中取一次数据需要次访问内存。

A 1 B 2 C 3 D 4

19. 在Hoare 管程中，假设进程P 正在使用管程，当P 执行唤醒Signal 操作时，其含义是 **C** 。

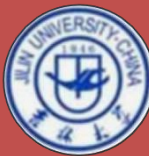
A Signal and continue B Signal and leave
C Signal and urgent wait D 以上都不对

一、单选



20. 某计算机系统中有8 台打印机，由K 个进程竞争使用，每个进程最多需要3 台打印机。该系统可能会发生死锁的K 的最小值是 **C** 。

A 2 B 3 C 4 D 5



二、实时调度

设有周期性实时任务集合如表1 所示，用最早截止期优先算法是否可以调度？如果可以调度，请画出前50 个时间单位相应的Gantt 图。

表1·周期性实时任务

任务	发生周期 T_i	处理时间 C_i
A	30	10
B	40	15
C	50	5

答：同作业一.3

三、死锁



考虑某个系统在下表时刻的状态

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	0	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	0	1	4	0	6	5	6				

使用银行家算法回答下面的问题：

- (1) Need 矩阵是怎样的？（5 分）
- (2) 系统是否处于安全状态？如安全，请给出一个安全序列。（5 分）
- (3) 若从进程 P1 发来一个请求 (1, 4, 2, 0)，这个请求是否可以被满足，如可以，请给出安全序列。（5 分）

三、死锁



考虑某个系统在下表时刻的状态

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	0	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	0	1	4	0	6	5	6				

(1) Need 矩阵

$$Need = Max - Allocation = \begin{bmatrix} 0,0,1,2 \\ 1,7,5,0 \\ 2,3,5,6 \\ 0,6,5,6 \end{bmatrix} - \begin{bmatrix} 0,0,1,2 \\ 1,0,0,0 \\ 1,3,5,4 \\ 0,0,1,4 \end{bmatrix} = \begin{bmatrix} 0,0,0,0 \\ 0,7,5,0 \\ 1,0,0,2 \\ 0,6,4,2 \end{bmatrix}$$

三、死锁



(1) Need 矩阵

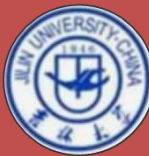
$$Need = Max - Allocation = \begin{bmatrix} 0,0,1,2 \\ 1,7,5,0 \\ 2,3,5,6 \\ 0,6,5,6 \end{bmatrix} - \begin{bmatrix} 0,0,1,2 \\ 1,0,0,0 \\ 1,3,5,4 \\ 0,0,1,4 \end{bmatrix} = \begin{bmatrix} 0,0,0,0 \\ 0,7,5,0 \\ 1,0,0,2 \\ 0,6,4,2 \end{bmatrix}$$

(2) 安全性分析: $work = available = (1, 5, 2, 0)$

- ① $Need[0] < work$, 更新 $work = (1, 5, 3, 2)$, $Finish[0] = true$;
- ② $Need[2] < work$, 更新 $work = (2, 8, 8, 6)$, $Finish[2] = true$;
- ③ $Need[1] < work$, 更新 $work = (3, 8, 8, 6)$, $Finish[1] = true$;
- ④ $Need[3] < work$, 更新 $work = (3, 8, 9, 10)$, $Finish[3] = true$;

安全序列 $\langle P0, P2, P1, P3 \rangle$

(3) $Request(P1) = (1, 4, 2, 0)$ 不小于 $Need(P1) = (0, 7, 5, 0)$;
 $Request(P1) = (1, 4, 2, 0) < Available(P1) = (1, 5, 2, 0)$,
故系统处于不安全状态。



四、存储管理

系统采用一级页表的虚拟页式存储管理方式，进程的页表空间由系统动态分配，页框大小为2KB，每个进程分配的页框数固定为3 页，采用局部置换策略，置换策略采用时钟算法（假设“时钟”指针指向页表中刚装入页面的页表表项）。主存空闲区管理采用空闲页面表的结构，存储分配采用最先适应算法。空闲页面表如下（表中数字均为十进制）

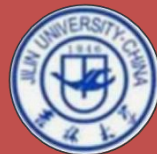
在上述条件下，系统创建了长度为9KB 的进程P。问题：

- (1) 进程 P 的PCB 中，其页表首址和页表长度的值分别是多少？（假设每个页表项占 4B，页表首址仅给出物理页框号即可）（5 分）
- (2) 当进程P 初始运行，依次访问：逻辑页 4、逻辑页 0、逻辑页 2，写出进程P 的页表内容。（5 分）
- (3) 在（2）的基础上，写出P 访问逻辑地址1AC2H 后P 的页表内容，该逻辑地址地址映射的物理地址是多少？（5 分）

空闲页面表

下标	首页框号	页框个数
0	55	2
1	63	1
2	82	3
3	95	20
4		0

四、存储管理



(1) 进程 P 的PCB 中，其页表首址和页表长度的值分别是多少？（假设每个页表项占 4B，页表首址仅给出物理页框号即可）（5 分）

答：系统创建进程P，要申请一页的页表空间，根据空闲页框表和最先适应分配算法，进程P 的页表空间分配的页框号为55，则P 的PCB 中

页表首址=55 号页框

进程P 长度为9KB，则

进程P 的逻辑页个数= $9\text{KB} \div 2\text{KB} = 5$ ，即页表长度=5；

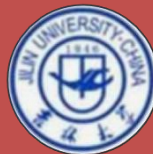
四、存储管理



(2) 当进程P 初始运行，依次访问：逻辑页 4、逻辑页 0、逻辑页 2，写出进程P 的页表内容。（5 分）

答：当进程P 初始运行，依次访问：逻辑页4、逻辑页0、逻辑页2，由于系统已经把55号页框分配给 P的页表，则系统给逻辑页面 4、 0、 2分配的页框号依次为： 56 63 82。则进程 P的页表如下：

逻辑页号	页框号	内外标志	访问标志
0	63	1	1
1	—	0	—
2	82	1	1
3	—	0	—
4	56	1	1



四、存储管理

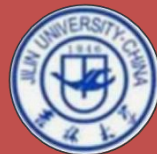
(3) 在(2)的基础上, 写出P 访问逻辑地址1AC2H后P的页表内容, 该逻辑地址地址映射的物理地址是多少? (5 分)

逻辑地址 1AC2H=0001,1010,1100,0010B, 由于页框大小为 2KB=2¹¹, 该逻辑地址访问逻辑页面为 3, 缺页; 进程固定分配 3 个页框, 逻辑页面 2 刚装入内存, 时钟指针指向页表中页面 2 的表项, 各访问位均为 1, 淘汰页面 2; 则访问逻辑地址 1AC2H 后 P 的页表内容如下:

逻辑页号	页框号	内外标志	访问标志
0	63	1	0
1	—	0	—
→ 2	82	0	0
3	82	1	1
4	56	1	0

页框号 82=1010010B, 把该二进制页框号替换逻辑地址中的逻辑页号 00011, 得 0010,1001,0010,1100,0010B=292C2H, 即逻辑地址 1AC2H 映射的物理地址是 292C2H。

五、信号量和 PV 操作

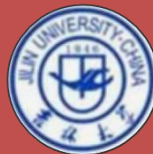


设系统有多个生产者进程向 1 个缓冲区不断发消息， n 个消费者进程从该缓冲区取消息。该缓冲区大小只能存放 1 条消息，并且对每条放入缓冲区的消息，所有消费者必须都接收 1 次，生产者才可以继续向缓冲区发消息。

问题：用信号量和 P、V 操作完成生产者进程和消费者进程发消息的正确描述。要求：

- (1) 给出信号量变量的定义、初值及其含义；
- (2) 实现对缓冲区及消息操作的互斥与同步；
- (3) 用类 C 语言伪代码描述。

五、信号量和 PV 操作



(1) 信号量变量定义:

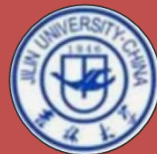
```
semaphore  mutex;    //初值为 1，用于缓冲区读、写互斥；  
  
semaphore  read_n[n]; //初值均为 1；用于生产者判断 n 个消费者均读完消息；  
  
semaphore  write_n[n]; //初值均为 0；用于生产者写入消息后，与 m 个消费者同步；  
  
message    buffer;    //假设 message 为消息类型，buffer 为缓冲区；
```

(2) 生产者、消费者进程描述:

生产者进程:

```
void producer( )  
{ int i; message ms;  
  while(1) {  
    ms=produce(); // “生产” 一条消息;  
    for(i=0;i<n;i++) P(&read_n[i]); //所有消费者都读完上次发的消息?  
                                     //如有消费者还没读消息，则生产者等待。  
    P(&mutex); write(buffer,ms); V(&mutex); //把消息 ms 写入缓冲区 buffer;  
    for(i=0;i<m;i++) V(&write_n[i]); //通知所有消费者，已经把消息写入缓冲区;  
  }  
}
```

五、信号量和 PV 操作



消费者进程:

```
void consumer(process_id k)    //k 为消费者进程号,  $0 \leq k \leq n-1$ ;  
{ int i; message ms;  
  while(1) {  
    P(&write_n[k]); //生产者已发消息? 如没发, 则等待;  
    P(&mutex); ms=read(buffer); V(&mutex); //把缓冲区 buffer 中消息读到 ms;  
    V(&read_m[k]); //通知生产者: 消费者进程 k 已经读完缓冲区中的消息;  
    consume(ms) //消费者“消费”消息 ms;  
  }  
}
```



谢谢！